PROYECTO NEXUS

Guía de Exposición y Documentación Técnica de Arquitectura

Este documento contiene el desglose detallado para la exposición del proyecto NEXUS. Cada sección contiene el contenido sugerido para las diapositivas, el guion explicativo recomendado y la ubicación exacta de los componentes en el código fuente.

TEMA 1: Autenticación con Firebase

📌 Qué colocar en la Diapositiva:

- Motor: Firebase Authentication (firebase/auth).
- Mecanismos: Registro (createUserWithEmailAndPassword), Login (signInWithEmailAndPassword), Logout (signOut).
- Persistencia Local: Integración con @react-native-async-storage/async-storage en React Native para no perder la sesión tras reiniciar la app.
- Estado Reactivo Global: AuthContext centraliza el listener onAuthStateChanged que conmuta automáticamente entre pantallas públicas y privadas.

🎙️ Qué decir durante la Exposición:

> *Para la seguridad y gestión de identidades implementamos Firebase Authentication. La inicialización se realizó en firebaseConfig.js, donde configuramos la persistencia mediante AsyncStorage en entornos móviles para mantener activa la sesión aunque el usuario cierre la aplicación.*
>
> *Todo el estado de autenticación se gestiona de forma global y reactiva a través de AuthContext.js con el listener onAuthStateChanged. Esto significa que cuando el usuario inicia sesión en LoginScreen o crea su cuenta en RegisterScreen, no necesitamos forzar manualmente la navegación; App.js detecta automáticamente si existe un objeto user activo y conmuta de inmediato entre las pantallas públicas y el menú principal de la aplicación.*

📂 Ubicación exacta en el Código:

Función / Componente	Archivo y Líneas
Configuración y persistencia	firebaseConfig.js (Líneas 20-36)
Manejador de estado global	context/AuthContext.js (Líneas 14-26)
Inicio de sesión	screens/LoginScreen.js (Líneas 20-36)
Registro de nuevo usuario	screens/RegisterScreen.js (Líneas 44-52)
Navegación protegida reactiva	App.js (Líneas 109-126)

TEMA 2: Conexión y Persistencia con Cloud Firestore

📌 **Qué colocar en la Diapositiva:**

- Base de datos: Cloud Firestore (NoSQL orientada a colecciones y documentos).
- Colección 'usuarios': Guarda nombre, nombre preferido, email y fecha de creación.
- Colección 'recordatorios': Alarmas y recordatorios asociados al UID del usuario.
- Sincronización en tiempo real: Uso de onSnapshot para actualizar la UI instantáneamente.
- Operaciones CRUD: addDoc, setDoc, updateDoc, deleteDoc y filtros con where('uid', '==', user.uid).

🎙️ **Qué decir durante la Exposición:**

La persistencia de los datos del asistente se realiza con Cloud Firestore. Al registrar un usuario en RegisterScreen, se crea un documento en la colección 'usuarios' usando su identificador único user.uid como clave primaria.

Para la gestión de recordatorios y alarmas creamos el RemindersContext.js. Aquí se ejecuta una consulta con query() filtrando únicamente los datos del usuario actual con where('uid', '==', user.uid). En lugar de hacer peticiones estáticas periódicas, usamos onSnapshot, lo que permite que cualquier creación con addDoc, actualización con updateDoc o eliminación con deleteDoc se sincronice en tiempo real con la interfaz gráfica de HomeScreen y RemindersScreen.

📂 **Ubicación exacta en el Código:**

Función / Componente	Archivo y Líneas
Instancia de base de datos	firebaseConfig.js (Línea 37)
Almacenamiento de perfil	screens/RegisterScreen.js (Líneas 53-60)
Lectura en tiempo real del perfil	screens/HomeScreen.js (Líneas 30-41)
Suscripción reactiva a recordatorios	context/RemindersContext.js (Líneas 74-98)
Funciones de creación, edición y borrado	context/RemindersContext.js (Líneas 135-191)

TEMA 3: Implementación de Notificaciones Locales

📌 **Qué colocar en la Diapositiva:**

- Librería: expo-notifications (con arquitectura modular).
- Canales Android (Channels): 'nexus-recordatorios' (prioridad alta) y 'nexus-alarmas' (prioridad máxima con bypassDnd activado).
- Triggers Temporales: Notificaciones programadas a futuro tipo SchedulableTriggerInputTypes.DATE.

- Sincronización de IDs: El ID de la notificación se almacena en Firestore para permitir cancelaciones y reprogramaciones.
- Compatibilidad: Aislamiento dinámico para operar limpiamente en Expo Go y Development Builds.

🎙️ **Qué decir durante la Exposición:**

Las notificaciones del sistema están centralizadas en services/notifications.js. Se crearon dos canales dedicados para Android: uno de prioridad regular para recordatorios y otro de prioridad máxima para alarmas, el cual tiene habilitado el traspaso del modo No Molestar (bypassDnd).

Cuando el usuario agenda una tarea o alarma, scheduleReminderNotification calcula la fecha exacta y la envía al planificador del sistema operativo con scheduleNotificationAsync. El ID generado por la notificación se almacena junto al documento en Firestore, lo que nos permite cancelar o reprogramar la notificación en el sistema operativo si el usuario borra la alarma o la marca como completada.

📂 **Ubicación exacta en el Código:**

Función / Componente	Archivo y Líneas
Definición y canales de notificación	services/notifications.js (Líneas 22-46)
Programación y cancelación horaria	services/notifications.js (Líneas 74-115)
Reconciliación y sincronización con Firestore	context/RemindersContext.js (Líneas 100-127)
Inicialización al montar la app	App.js (Líneas 128-135)

TEMA 4: Respuesta por Voz de la IA (Text-to-Speech)

📌 **Qué colocar en la Diapositiva:**

- Motor Text-to-Speech: expo-speech.
- Parámetros de voz: Idioma español (es-ES), tono (pitch: 1.0), velocidad (rate: 1.0).
- Prevención de colisiones: Speech.stop() antes de cada emisión para evitar solapamientos.
- Integración en Chat: La función responderNexus() muestra el texto en pantalla y emite el audio simultáneamente.
- Configurabilidad: Interruptor interactivo para silenciar o activar la voz según la preferencia del usuario.

🎙️ **Qué decir durante la Exposición:**

Para que el asistente NEXUS pudiera responder verbalmente, integramos la síntesis de voz en services/voice.js utilizando expo-speech.

Diseñamos la función hablar(texto), la cual detiene cualquier reproducción anterior en cola para evitar que

las frases se superpongan y sintetiza la respuesta con acento en español y modulación natural.

En ChatScreen.js, cada vez que la lógica del asistente genera una respuesta de confirmación, saludo o error, se ejecuta responderNexus(texto), que simultáneamente publica la burbuja en el chat y emite el audio si el usuario tiene el interruptor de voz habilitado.

📂 Ubicación exacta en el Código:

Función / Componente	Archivo y Líneas
Módulo de síntesis de voz (hablar / detener)	services/voice.js (Líneas 30-51)
Emisión de audio ligada al chat	screens/ChatScreen.js (Líneas 66-69)

TEMA 5: Reconocimiento y Procesamiento de Órdenes por Voz

📌 Qué colocar en la Diapositiva:

- Speech-to-Text: expo-speech-recognition para captura de audio con soporte interimResults y eventos finales.
- Motor de PLN propio: services/assistant.js (sin dependencias externas pesadas).
- Detección de Intenciones: crear, eliminar, listar, hora, ayuda, saludo.
- Extracción temporal inteligente: Soporte para 'en 20 minutos', 'mañana a las 3 pm', 'el próximo viernes'.
- Limpieza de lenguaje natural: Filtra muletillas como 'oye nexus', 'por favor', 'recuérdame'.

🎙️ Qué decir durante la Exposición:

La captura de órdenes por voz combina dos capas fundamentales:

1. Reconocimiento del habla: En services/voice.js, mediante expo-speech-recognition, abrimos el micrófono del dispositivo, solicitamos los permisos requeridos y escuchamos en streaming. Cuando el usuario termina de hablar (isFinal), el texto capturado se envía al procesador.

2. Procesamiento de Lenguaje Natural: En services/assistant.js desarrollamos el motor lógico procesarOrden(). Este analiza el texto, normaliza las palabras y extrae tanto la intención como las entidades de tiempo gracias a resolverFecha(). Por ejemplo, si el usuario dice: 'NEXUS, por favor recuérdame tomar agua en 15 minutos', el sistema extrae automáticamente el título 'Tomar agua', calcula la fecha sumando los minutos a la hora actual y ejecuta la creación del recordatorio en Firestore y la alarma en el sistema operativo.

📂 Ubicación exacta en el Código:

Función / Componente	Archivo y Líneas
Captura de micrófono y streaming	services/voice.js (Líneas 64-152)

Botón de micrófono e interacción de UI	screens/ChatScreen.js (Líneas 143-178)
Extracción de horas y fechas relativas	services/assistant.js (Líneas 93-226)
Motor de intenciones (procesarOrden)	services/assistant.js (Líneas 305-374)

TABLA RESUMEN DE ARQUITECTURA

Módulo	Archivos Clave	Tecnologías
Autenticación	firebaseConfig.js, AuthContext.js, LoginScreen.js, RegisterScreen.js	Firebase Auth, AsyncStorage
Base de Datos	firebaseConfig.js, RemindersContext.js, RegisterScreen.js, HomeScreen.js	Cloud Firestore (onSnapshot, addDoc)
Notificaciones	services/notifications.js, RemindersContext.js, App.js	expo-notifications, Android Notification Channels
Voz IA (Salida)	services/voice.js, ChatScreen.js	expo-speech (Text-to-Speech es-ES)
Órdenes de Voz (Entrada)	services/voice.js, services/assistant.js, ChatScreen.js	expo-speech-recognition, Regex PLN Engine